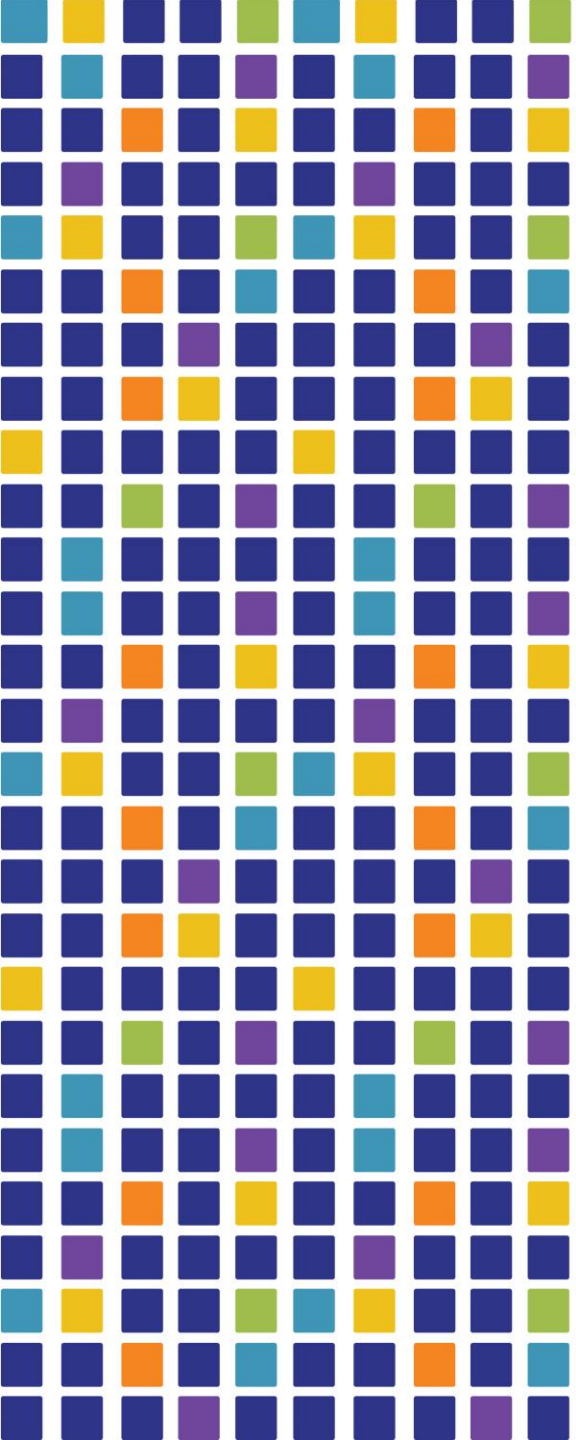




Chapter – 4

Managing Process

Managing Processes

When you type a command name, that program is run and a process is created for it. This chapter includes identifying processes, manipulating foreground and background processes, killing processes, and adjusting process priorities.

Understanding the Kernel: The First Process

The Linux kernel is at the heart of every Linux system. You can use the **uname** command, which takes several options to display information:

- **Node Name** The `-n` or `--nodename` option displays the system's node name; that is, its network hostname.
- **Kernel Name** The `-s` or `--kernel-name` option displays the kernel name, which is Linux on a Linux system.
- **Kernel Version** You can find the kernel version with the `-v` or `--kernel-version` option.
- **Machine** The `-m` or `--machine` option returns information about your machine. This is likely to be a CPU code, such as `i686` or `x86_64`.
- **Processor** Using the `-p` or `--processor` option may return information about your CPU, such as the manufacturer, model, and clock speed; in practice, it returns unknown on many systems.
- **Hardware Platform** Hardware platform information is theoretically returned by the `-i` or `--hardware-platform` option, but this option often returns unknown.
- **OS Name** The `-o` or `--operating-system` option returns the OS name—normally GNU/ Linux for a Linux system.
- **Print All Information** The `-a` or `--all` option returns all available information.

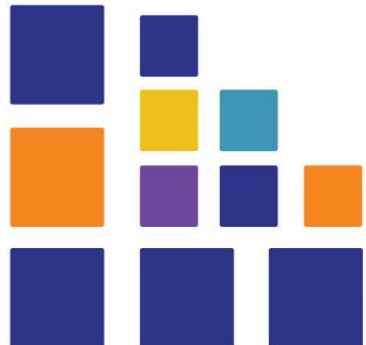
Understanding Foreground and Background Processes

One of the most basic process-management tasks is to control whether a process is running in the foreground or the background; that is, whether it's monopolizing the use of the terminal from which it was launched. Normally, when you launch a program, it takes over the terminal, preventing you from doing other work in that terminal.

A process that connects to the terminal is called a foreground job. A job is said to be in the **foreground** because it can communicate with the user via the screen and the keyboard.

A process that disconnects from the terminal and cannot communicate with the user is called a **background** job. If the background job requires interaction with the user, it will stop and wait until establishing a connection to the terminal.

Editing a file with the NEdit GUI editor by typing **nedit myfile.txt**, you can type **nedit myfile.txt &**. This command launches the nedit program in the background from the start, leaving you able to control your xterm window for other tasks.



Examining Process Lists

One of the most important tools in process management is **ps**. This program displays processes' status (hence the name, ps). This can be particularly critical when the computer isn't working as it should be—for instance, if it's unusually slow. The ps program supports an unusual number of options. Likewise, interpreting ps output can be tricky because so many options modify the program's output.

The syntax for ps is : **ps [options]**

Some of the more useful ps features include the following:

- Display Help The --help option summarizes some of the more common ps options.
- Display All Processes By default, ps displays only processes that were run from its own terminal (xterm, text-mode login, or remote login). The -A and -e options cause it to display all of the processes on the system, and x displays all processes owned by the user who gives the command. The x option also increases the amount of information that's displayed about each process.
- Display One User's Processes You can display processes owned by a given user with the -u user, U user, and --User options. The user variable may be a username or a user ID.
- Display Extra Information The -f, -l, j, l, u, and v options all expand the information provided in the ps output. Most ps output formats include one line per process, but ps can display enough information that it's impossible to fit it all on one 80-character line.
- Display Process Hierarchy The -H, -f, and --forest options group processes and use indentation to show the hierarchy of relationships between processes. These options are useful if you're trying to trace the parentage of a process.
- Display Wide Output The ps command output can be more than 80 columns wide. Normally, ps truncates its output so that it will fit on your screen or xterm. The -w and W options tell ps not to do this, which can be useful if you direct the output to a file, as in `ps w > ps.txt`. You can then examine the output file in a text editor that supports wide lines.

Interpreting ps Output

\$ ps u rodsmith

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
rodsmith	19221	0.0	1.5	4484	1984	?	S	May07 0:01	dfm	
rodsmith	2451	0.0	0.8	1856	1048	pts/3	S	16:13 0:00	-bash	
rodsmith	2496	0.2	3.2	6232	4124	?	S	16:17 0:00	/opt/kd	
rodsmith	2498	0.0	0.8	1860	1044	pts/1	S	16:17 0:00	bash	
rodsmith	2505	0.1	2.6	4784	3332	pts/1	S	16:17 0:00	nedit	
rodsmith	2506	0.0	0.7	2124	1012	?	S	16:17 0:00	/bin/cs	
rodsmith	2544	0.0	1.0	2576	1360	?	S	16:17 0:00	xeyes	
rodsmith	2556	0.0	0.7	2588	916	pts/3	R	16:18 0:00	ps u	

USER – Username

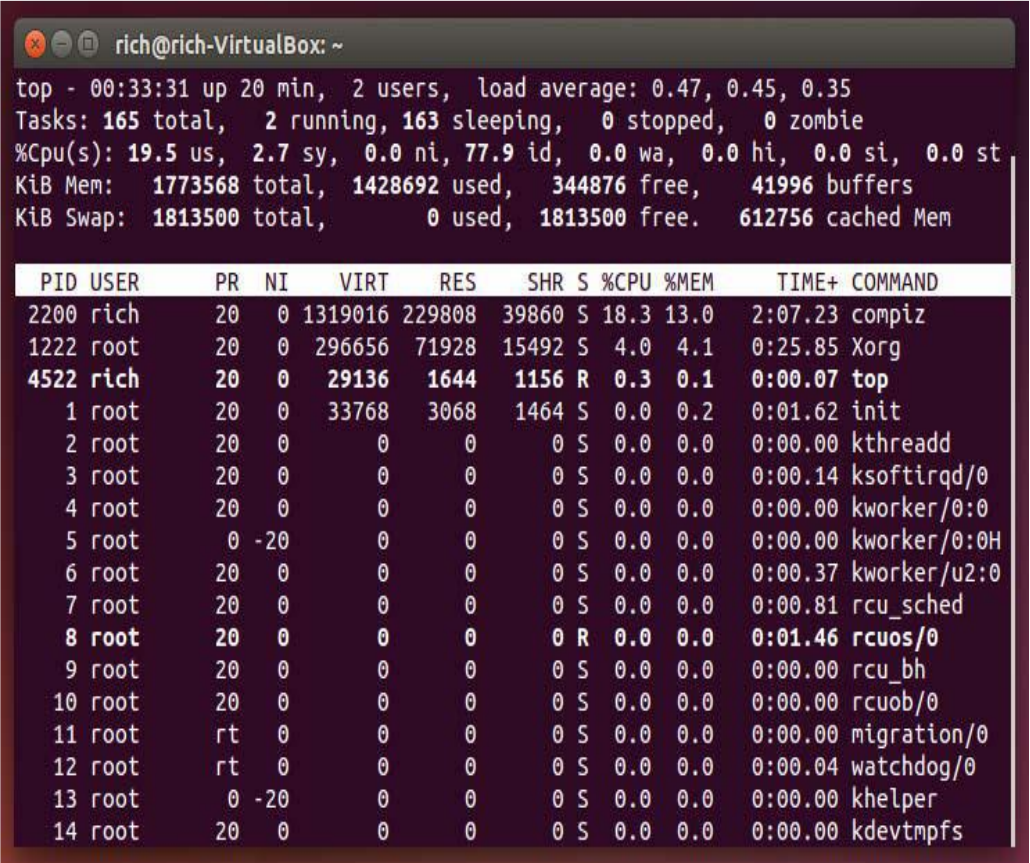
PID - Process ID

CPU - CPU time and %

MEM – Memory Use

VSZ - Virtual set size

- RSS - RSS is resident set size (the memory used by the program and its data)
- TTY - The teletype (TTY) is a code used to identify a terminal
- STAT - Status
- START – Start Time
- TIME - CPU time used
- COMMAND – Which command is working



jobs: Processes Associated with Your Session

The **jobs** command displays minimal information about the processes associated with the current session. In practice, jobs is usually of limited value, but it does have a few uses. One of these is to provide job ID numbers. These numbers are conceptually similar to PID numbers, but they're not the same.

pgrep: Finding Processes

The **pgrep** command was introduced in the Solaris operating system, but it has been ported to the open-source world and is becoming more popular in Linux. It allows you to perform simple searches within the process list; similar to piping the **ps** command output to the **grep** command.

You can search for processes based on the username, user ID, or group ID as well as any type of regular expression pattern:

```
$ pgrep -u root cron
```

```
812
```

```
$
```

This example searches for a process named cron, run by the root user account. Notice that the **pgrep** command only returns the process ID value of the processes that match the query.



Managing Process Priorities

Sometimes, you may want to prioritize your programs' CPU use. For instance, you may be running a program that's very CPU intensive but that will take a long time to finish its work, and you don't want that program to interfere with others that are of a more interactive nature. Use **nice** to launch a program with a specified priority or use **renice** to alter the priority of a running program.

You can assign a priority to nice in any of three ways: by specifying the priority preceded by a dash (this works well for positive priorities but makes them look like negative priorities), by specifying the priority after the `-n` parameter, or by specifying the priority after the `--adjustment=` parameter. In all cases, these parameters are followed by the name of the program you want to run:

nice [argument] [command [command-arguments]]

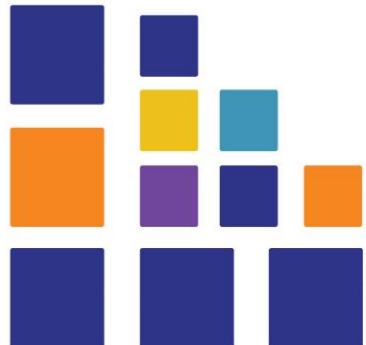
For instance, the following three commands are all equivalent:

```
$ nice -12 number-crunch data.txt
```

```
$ nice -n 12 number-crunch data.txt
```

```
$ nice --adjustment=12 number-crunch data.txt
```

All three of these commands run the number-crunch program at priority 12 and pass it the data.txt file.



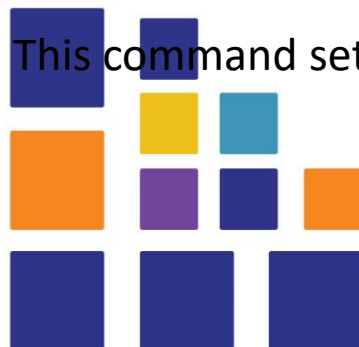
If you've found that a running process is consuming too much CPU time or it is being swamped by other programs and so should be given more CPU time, you can use the `renice` program to alter its priority without disrupting the program's operation. The syntax for `renice` is as follows:

`renice priority [[-p] pids] [[-g] pgrps] [[-u] users]`

You must specify priority, which takes the same values this variable takes with `nice`. In addition, you must specify one or more PIDs (`pids`), one or more group IDs (`pgrps`), or one or more usernames (`users`). In the latter two cases, `renice` changes the priority of all programs that match the specified criterion—but only root may use `renice` in this way. Also, only root may increase a process's priority. If you give a numeric value without a `-p`, `-g`, or `-u` option, `renice` assumes the value is a PID. You may mix and match these methods of specification. For instance, you might enter the following command:

`# renice 7 16580 -u pdavison tbaker`

This command sets the priority to 7 for PID 16580 and for all processes owned by `pdavison` and `tbaker`.



Killing Processes

Sometimes, reducing a process's priority isn't a strong enough action. A program may have become totally unresponsive, or you may want to terminate a process that shouldn't be running. In these cases, the **kill** command is the tool to use. The syntax for kill is as follows:

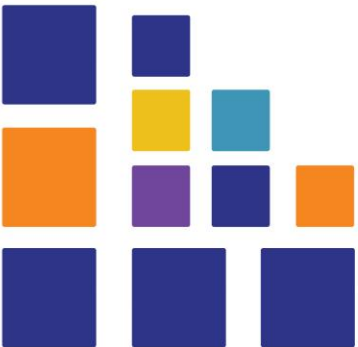
```
kill -s signal pid
```

The variant on kill is killall, which has the following form:

```
killall [options] [--] name [...]
```

This command kills a process based on its name rather than its PID number. For instance, **killall vi** kills all of the running processes called vi.

```
$ killall -i vi  
Kill vi(13211) ? (y/n) y  
Kill vi(13217) ? (y/n) n
```



End of Chapter 4

